Sujet du Projet Système et Réseau (ETE7-PROJ1) :

Implémentation d'une bibliothèque de Mémoire Partagée Distribuée (DSM)

Fadwa Abakarim, Guillaume Mercier, Philippe Swartvagher

fadwa.abakarim@bordeaux-inp.fr,
guillaume.mercier@bordeaux-inp.fr,
philippe.swartvagher@bordeaux-inp.fr

2026

1 Thématique du projet

1.1 Notion d'application répartie

Ce projet sera l'occasion de vous familiariser avec l'univers des applications *distribuées et réparties*. Il s'agit d'applications dont la structure (ou l'architecture) est basée sur un ensemble d'entités (par exemple des processus) s'exécutant généralement sur des machines physiquement différentes. Par exemple, vous avez vu en cours de programmation réseau des applications de type client-serveur, où les différents clients dialoguent avec le serveur. Il est aussi parfois possible que les différents clients communiquent les uns avec les autres. Cependant, il existe d'autres sortes d'applications réparties qui ne sont pas basées sur une architecture client-serveur. Dans ce cas, les différentes entités de l'application communiquent les unes avec les autres.

1.2 Communications dans les applications réparties

Ces communications ont pour but l'échange de données ou encore les synchronisations éventuelles entre les différents morceaux de l'application répartie. Elles utilisent le réseau d'interconnexion sous-jacent (p. ex. Ethernet) en s'appuyant sur une interface de programmation disponible. À titre d'exemple, le cours de programmation réseau aura été l'occasion pour vous de découvrir et manipuler l'interface socket qui existe dans les systèmes de type UNIX.

Cependant, des alternatives existent pour effectuer les communications se produisant au sein d'une application distribuée. Ainsi, il est possible de mettre en place des mécanismes utilisant le réseau sous-jacent *implicitement*, c'est-à-dire que cette utilisation se fait de façon transparente par l'application. Par exemple, il est envisageable de mettre en place un système de **mémoire partagée distribuée** (*Distributed Shared Memory*) pour gérer ces communications. Du point de vue de l'application, les communications sont effectuées en s'échangeant des données via une zone mémoire classique, à ceci près que **cette zone est partagée par tous les processus**. Pratiquement, l'application utilise une bibliothèque qui se charge des communications nécessaires via l'interface réseau des sockets. Un tel système demande de mettre en place une **infrastructure** pour permettre ce type de communications puisque si les processus sont situés sur des machines

physiquement distinctes, il est bien évident qu'ils sont dans l'incapacité de se partager véritablement de la mémoire.

1.3 Déploiement d'applications réparties

Par ailleurs, le **déploiement** d'une application répartie pose aussi son lot de problèmes : s'il est théoriquement possible de tout faire manuellement, cette solution est pratiquement trop longue et pénible pour être utilisable. (p. ex. lancer 100 processus (quasi)-simultanément sur autant de machines distinctes, échanger les informations de connexion entre tous les processus, etc.). Il est donc courant d'utiliser un programme annexe, permettant de faciliter ce déploiement en automatisant les étapes redondantes et la récupération/génération/diffusion des informations nécessaires.

2 Logistique du projet

2.1 Productions attendues

Les productions attendues concernant ce projet sont les suivantes :

Production 1 : la création et le développement de l'infrastructure nécessaire pour un déploiement facile d'une application distribuée basée sur un système de mémoire partagée distribuée. Cette infrastructure prendra la forme d'un **programme annexe** appelé le **lanceur d'applications** (programme `dsmexec`).

Production 2 : la création et le développement de l'infrastructure nécessaire pour permettre des communications via de la mémoire partagée entre les différents processus d'une application répartie. Cette infrastructure prendra la forme d'une **bibliothèque** (`libdsm.a`) appelée par les applications (cf programmes d'exemple).

Les séances de projet sont organisées en **deux phases distinctes** (Phase 1 suivie de la Phase 2). Chacune de ces phases est dédiée à la réalisation d'un des deux résultats décrits ci-dessus. La Phase 2 ne débutera qu'une fois la Phase 1 achevée afin que vous ayez la possibilité de travailler sur l'ensemble des aspects du projet.

2.2 Organisation des phases

La ventilation des séances entre les deux phases est la suivante :

- la Phase 1 sera couverte par les 11 premières séances (de 1h20, soit 14h40 au total).
  Par ailleurs, la toute première séance sera consacrée à :
  - la prise en main (relecture) du sujet
  - la configuration de vos dépôts Git respectifs (cf ci-dessous)
  - la création des clefs ssh
  - la définition des variables d'environnement nécessaires pour le travail demandé.
  - ⇒ Voir la section 7 en toute fin de sujet pour les tâches à accomplir et les détails.
- la Phase 2 sera couverte par les 9 dernières séances (soit 12h au total)

À la fin de la dernière séance de chaque phase, nous mènerons des tests afin de déterminer si le code produit est fonctionnel et respecte le cahier des charges. Un rapport d'exécution vous sera communiqué pour chaque série de tests.

Un planning indicatif des tâches et sous-tâches est fourni en fin de document (cf Section 6.2).

2.3 Environnement de travail

Étant donné que le travail consiste en l'élaboration d'une infrastructure destinée à faciliter le développement et le déploiement d'applications réparties, vous allez devoir utiliser plusieurs machines. Cependant, des précisions s'imposent :

— En ce qui concerne le développement, une seule machine est nécessaire. Par conséquent, il est possible de développer intégralement le projet sur une machine de l'ENSEIRB ou sur vos machines personnelles.
— En ce qui concerne les tests de votre projet, il est aussi possible de les effectuer sur une unique machine (de l'école ou personnelle). Cependant, le but étant de réaliser une infrastructure pour le déploiement et l'exécution d'applications distribuées il est impératif de tester votre travail sur plusieurs machines de l'école. À cela, deux raisons : d'une part il est possible que des bugs puissent se déclencher quand vous allez passer d'un environnement mono-machine à un environnement multi-machines. Secondement, les tests de fin de phase seront menés exclusivement sur un ensemble de machines de l'école.
— Il ne faut pas utiliser la passerelle ssh de l'école (machine `ssh.enseirb-matmeca.fr`) pour travailler (développement, tests ou déploiement) ;
— De même il ne faut pas utiliser la machine `travail64` pour effectuer des tests de déploiement car il n'est pas possible de se connecter depuis cette machine vers d'autres machines de l'école.
— Une liste des machines utilisables vous sera fournie dans vos dépôts Git lors de la première phase. **Cette liste est assez longue (mais probablement pas exhaustive) ; vous ferez attention à ne pas tou(te)s utiliser les mêmes machines afin de ne pas trop les charger.**

Soyez vigilant(e)s à faire le ménage sur les machines que vous utilisez lors des séances : vérifiez en fin de séance que vos processus ne continuent pas de s'exécuter.

2.4 (Res)sources de départ

Liste des ressources fournies pour le projet :
— Le sujet dont vous êtes en train de prendre connaissance. il est impératif de le lire attentivement. Il contient un nombre conséquent d'informations et d'indications qui vont vous permettre de mener à bien le projet.
— Une base de code pour la Phase 1 : consultez la section 4.5 pour plus de détails. Cette ressource est disponible dès le début du projet.
— Une base de code pour la Phase 2 : consultez la section 5.3 pour plus de détails. Cette ressource vous sera fournie quand débutera la Phase 2 (via une mise à jour de vos dépôts Git).

3 Fonctionnement de la mémoire partagée distribuée

Pour que les différents processus d'une application distribuée puissent communiquer entre-eux, ils vont avoir besoin d'une plage d'adresses particulière dans leur espace d'adressage respectif. Cette **plage d'adresses est identique pour tous les processus** et sera organisée en un ensemble de morceaux de taille fixe (4ko) appelés *pages mémoire*. Les processus vont donc manipuler ces pages, c'est-à-dire y lire et/ou y écrire des données. Cependant, un **unique processus sera le propriétaire** d'une page à un instant donné. Ce processus propriétaire peut accèder normalement à cette page, tandis que si les autres processus essayent d'y lire ou d'y écrire, cela provoquera une erreur d'accès mémoire (i.e un `segfault`). Quand cet évènement se produit, un signal (numéro `SIGSEGV`, 11) est envoyé au (thread du) processus qui a provoqué cette erreur. Le comportement par défaut d'un processus qui reçoit le signal `SIGSEGV` est normalement de se terminer.

Cependant, notre cas est différent. Comme cette page est **théoriquement accessible**, on considère que cette erreur n'en est pas vraiment une : le signal est pris en compte à l'aide d'un *traitant* adapté qui, au lieu de terminer le processus, va :

1. déterminer l'adresse qui a provoqué l'erreur (le *segfault*) et donc l'émission du signal ;
2. en déduire la page mémoire concernée, qui sera identifiée par un *numéro* ;
3. en déduire le processus actuellement propriétaire puisque un unique processus est le proprétaire de la page avec le numéro concerné ;
4. envoyer une requête **via le réseau** à ce processus pour qu'il envoie en retour la page au processus demandeur (ie. celui qui a provoqué l'erreur). Le processus demandeur devient donc le *nouveau propriétaire* de la page mémoire concernée ;
5. récupérer la page envoyée (via le réseau) par le processus propriétaire et la copier dans l'espace d'adressage du processus demandeur à l'emplacement idoine.

NB : Seules les erreurs de segmentation se produisant dans la zone de mémoire partagée seront rattrapées. Les autres erreurs de segmentation ne bénéficieront pas d'un traitement particulier.

L'actuel processus propriétaire devra :

1. mettre à jour ses informations concernant cette page (en particulier l'identité du nouveau processus propriétaire) ;
2. indiquer que cette page n'est plus accessible dans son espace d'adressage ;
3. envoyer la page-mémoire concernée au processus-demandeur.

Le processus demandeur devra :

1. recevoir la page via une socket dédiée (cf plus loin) et la copier dans son espace d'adressage ;
2. mettre à jour ses informations concernant cette page ;
3. reprendre son exécution : en ressortant du traitant, le processus réexécute automatiquement l'instruction qui a provoqué la faute. Cependant, comme ce processus possède désormais la page mémoire (elle est maintenant accessible dans son espace d'adressage), il peut travailler dessus sans que cela ne provoque une nouvelle erreur d'accès mémoire (*segfault*).

Vous réfléchirez à la manière dont ces informations doivent également être répercutées aux autres processus, afin que tous possèdent des informations cohérentes sur l'état du système.

4 Première phase : le lanceur d'applications `dsmexec`

Le programme `dsmexec` a pour fonction principale de **déployer** une application répartie. Avant d'examiner plus en détail le fonctionnement de ce programme, nous allons fixer les idées avec un exemple basique.

4.1 Un exemple basique de déploiement

Prenons comme exemple le programme `truc` que l'on souhaite déployer sur les trois machines suivantes :

1. `toto.pedago.ipb.fr`
2. `tata.pedago.ipb.fr`
3. `titi.pedago.ipb.fr`

Supposons ensuite que ce programme `truc` prend trois arguments : `arg1`, `arg2` et `arg3`. Dans ce cas de figure, le déploiement du programme sur les différentes machines sera effectué avec la ligne de commande suivante :

```
dsmexec machinefile truc arg1 arg2 arg3
```

Avec le fichier nommé `machinefile` qui contient les lignes :

```
toto.pedago.ipb.fr
tata.pedago.ipb.fr
titi.pedago.ipb.fr
```

Évidemment, cela suppose que le programme `truc` est disponible sur toutes les machines listées dans le fichier `machinefile`. Une hypothèse importante est que l'on suppose que **toutes les machines ont accès au même système de fichier**, ce qui résoud ce problème de disponibilité (pour une liste complète des hypothèses, voir la section 4.4).

4.2 Arguments de la commande `dsmexec`

Nous détaillons maintenant les arguments que le programme `dsmexec` accepte :

— **Premier argument :** un fichier contenant les noms des différentes machines sur lesquelles l'application distribuée doit être déployée.

— **Deuxième argument :** le nom du programme à déployer.

— **Arguments suivants :** les éventuels arguments supplémentaires du programme à déployer.

Précisions supplémentaires :

1. **L'ordre des arguments est fixe** : d'abord le nom du fichier de machines, puis le nom de la commande à déployer et enfin les éventuels arguments de cette commande.
2. Les deux premiers arguments (i.e nom de fichier de machines et nom de commande) sont **obligatoires**.
3. Le nom du fichier contenant les noms des machines est **quelconque**. Vous veillerez donc à ne pas coder « en dur » ce nom.

4.2.1 Formatage du fichier de machines

L'organisation du fichier de machines influe sur le comportement de l'application à déployer. Ce fichier est formaté de la façon suivante :

- Le **nombre de lignes** dans le fichier correspond au **nombre de processus** composant l'application distribuée.
- Chaque ligne contient **exactement** un nom de machine.
- L'ordre des machines listées dans le fichier induit la **numérotation** des différents processus. Plus précisément, les processus sont numérotés de façon contigüe de 0 à $n-1$ (si n processus sont déployés). Ainsi, le processus numéroté 0 sera lancé sur la machine dont le nom apparaît sur la première ligne du fichier, le processus 1 sera lancé sur la machine dont le nom apparaît sur la ligne suivante et ainsi de suite.
- Un nom de machine peut apparaître plusieurs fois dans ce fichier.
- Attention à gérer convenablement — c'est-à-dire ne pas les prendre en compte — les éventuelles lignes vides dans le fichier de machines (surtout en fin de fichier) car elles pourraient fausser le comportement du programme `dsmexec`.

On partira du principe que le **fichier est correctement structuré**, c'est-à-dire que nous ne vous demandons pas de gérer les cas où le fichier est mal formaté (à part le cas des lignes vides).

Enfin, le programme `dsmexec` peut tout à fait s'exécuter sur une machine qui n'est pas listée dans le fichier de machines pris en premier argument.

4.2.2 Restrictions concernant les noms des machines

Afin d'éviter certains problèmes au moment du déploiement d'une application répartie, nous vous demandons de respecter certaines restrictions au niveau de l'utilisation des noms de machines que vous allez écrire dans le fichier de de machines.

- le nom `localhost` ne doit pas être présent dans ce fichier ;
- il est préférable de mettre des *Fully qualified domain names (FQDN)* c'est-à-dire des noms incluant le nom de domaine complet. Si vous tapez la commande `hostname` dans un terminal, vous aurez le nom court de la machine sur laquelle vous vous trouvez tandis que la commande `hostname --long` vous donne le FQDN. Par exemple si je suis sur la machine `montag`, son nom court est `montag` tandis que son FQDN est `montag.pedago.ipb.fr`. NB : une configuration adéquate de ssh (cf paragraphe 7.2) peut vous éviter de mettre systématiquement les FQDN dans le fichier de machines.

Vous pourrez reprendre tels quels les noms indiqués dans le fichier (listant les machines utilisables) fourni lors de la Phase 1 (fichier `machines_list.txt`).
Rappel : répartissez-vous les machines pour ne pas utiliser tou(te)s les mêmes !

4.3 Rôles et fonctions du lanceur `dsmexec`

En fait, si le programme `dsmexec` a pour fonction principale de déployer des applications distribuées, il joue également deux autres rôles importants :

- centraliser et afficher les informations provenant des flux de sortie (i.e `stdout` et `stderr`) de tous les processus composant l'application répartie et déployés sur les machines distantes ;
- centraliser et rediffuser les informations nécessaires aux interconnexions de ces processus entre-eux (typiquement, le numéro de port de l'adresse IP — ou le nom — de la machine concernée).

Ces différents rôles et fonctions sont détaillés dans les paragraphes suivants.

4.3.1 Déploiement des processus d'une application répartie

Le déploiement doit se conformer aux points suivants :

1. **Emploi de `ssh` :** pour rappel, il est possible d'exécuter une commande sur une machine distante avec la commande `ssh`. Concrètement, la commande `ssh` crée un processus sur la machine-cible et ce processus exécute la commande passée en argument à `ssh`. Vous allez vous appuyer sur ce même mécanisme : afin de déployer de nouveaux processus sur les machines distantes, le programme `dsmexec` va utiliser `ssh` via un appel à une fonction de recouvrement de la famille `exec`. Ces nouveaux processus seront appelés les **processus distants**. Par conséquence, `dsmexec` va devoir créer un ensemble de **processus-enfants locaux** (i.e s'exécutant sur la même machine que lui) qui vont faire appel à `exec` pour exécuter la commande `ssh` avec des arguments pertinents. En effet, si `dsmexec` faisait directement appel à `ssh`, il ne pourrait créer qu'un seul processus sur une machine distante. **NB : il y a autant de processus-enfants locaux que de processus distants.**
2. **Numérotation des processus distants :** comme expliqué en section 4.2.1, si n processus sont déployés, alors ils sont numérotés de façon contigüe de 0 à $n-1$. C'est le programme `dsmexec` qui est chargé de transmettre aux processus distants leurs numéros respectifs. Plus exactement, cette numérotation est déduite par `dsmexec` de la structure du fichier de machines dont le nom lui est passé en premier argument.
3. **Déploiement d'une application quelconque :** en fait, on ne souhaite pas se limiter à des déploiements d'applications réparties utilisant la mémoire partagée distribuée (ie la `libdsm`). Un ajout important au cahier des charges est que **le programme `dsmexec` doit être capable de lancer tout type d'applications, de programmes ou de commandes**.

4.3.2 Centralisation de l'affichage des processus distants

Une fois le déploiement des processus distants effectué, le programme `dsmexec` est chargé de centraliser l'affichage des processus distants). Il faut donc trouver un moyen de récupérer cet affichage, qui est à l'origine effectué sur les flux de sortie des processus distants (ie `stdout` et `stderr`). cette récupération va se faire en deux étapes :

- **Première étape :** le flux des processus distants sera récupéré par les processus-enfants locaux *automatiquement* grâce à la commande `ssh`. Plus précisément, l'intégralité du flux `stdout` (resp. `stderr`) d'un processus distant donné est redirigé vers le flux `stdout` (resp. `stderr`) du processus local responsable de la création de ce processus distant. En conclusion : vous n'avez donc rien à faire de particulier pour cette étape car il s'agit du fonctionnement de base de la commande `ssh`;
- **Seconde étape :** il faut que le processus-parent `dsmexec` récupère l'affichage de tous ses processus-enfants locaux. Cette récupération sera effectuée à l'aide de tubes de communication : pour chaque type de flux de sortie (`stdout` et `stderr`) un tube est créé entre le processus-enfant local et le processus-parent `dsmexec`. Dans un tel fonctionnement, **les processus-enfants locaux doivent rediriger leur deux flux de sortie vers l'extrémité en écriture de chaque tube dédié.** (cf exercices du cours de programmation système sur les tubes).

Donc, le processus `dsmexec` doit récupérer (= lire) sur les différents tubes les flux de sortie de ses processus-enfants locaux, jusqu'à la terminaison du programme déployé.

Par ailleurs, le processus `dsmexec` doit afficher ces informations de sortie en utilisant un formatage particulier, comportant *a minima* le numéro de processus distant et la nature du flux (`stdout` ou `stderr`). Mis à part ces informations supplémentaires, l'affichage d'origine doit être respecté autant que faire se peut (sauts de ligne, espaces, etc.). Par exemple, en supposant que l'on exécute le programme `bidule` dont voici le code :

```
#include <stdio.h>

int main(int argc, char *argv[])
{
  fprintf(stdout,"Hello World!\n");
  return 0;
}
```

avec la ligne de commande suivante :

```
dsmexec machinefile bidule
```

Alors l'affichage de `dsmexec` pourra être formaté selon le modèle suivant :

```
[Proc 0 : toto.pedago.ipb.fr : stdout] Hello World !
[Proc 1 : tata.pedago.ipb.fr : stdout] Hello World !
[Proc 2 : titi.pedago.ipb.fr : stdout] Hello World !
```

Le formatage de l'affichage centralisé se présente ici sous la forme :

[*numéro de processus* : *nom de machine* : *type de flux de sortie*] *affichage du programme*

4.3.3 Récupération et diffusion des informations d'interconnexion

Comme indiqué en Section 3, les processus d'une application utilisant la mémoire partagée distribuée sont tous interconnectés — via des sockets TCP — afin de pouvoir s'échanger les pages mémoire accédées en lecture ou en écriture pendant l'exécution de l'application.

Deux processus peuvent s'interconnecter via une séquence `connect/accept` s'ils disposent des informations nécessaires, c'est-à-dire un couple :

< *adresse IP* (ou *nom de machine*) , *numéro de port de la socket d'écoute* >

1 — `dsmexec` en tant que serveur L'approche la plus simple repose sur le processus lanceur `dsmexec`, qui se charge de récupérer l'ensemble de ces données d'interconnexion puis de les diffuser à tous des processus de l'application. **Ce rôle implique que le processus `dsmexec` ouvre une socket dédiée à ces échanges à laquelle tous les processus distants doivent se connecter**. Ainsi, pour cet échange, le processus `dsmexec` jouera le rôle de serveur dont les processus applicatifs distants seront les clients.

Chaque processus applicatif distant doit donc ouvrir n sockets : $n - 1$ sockets pour les communications avec les autres processus applicatifs (inutile d'ouvrir une socket pour lui-même) plus une dernière socket pour les communications avec le programme `dsmexec`.

Cela signifie que les informations permettant aux processus distants de se connecter au processus `dsmexec` — adresse IP (ou le nom) de la machine sur laquelle s'exécute le processus `dsmexec` et le numéro de port de sa socket dédiée aux échanges — doivent êtres connues. **Ces informations seront communiquées sous la forme de variables d'environnement aux processus distants via la commande `ssh` au moment de leur déploiement.**

2 — Opacité des informations échangées **Par ailleurs, la nature de ces informations n'est connue que des processus applicatifs ; elles sont opaques pour le programme `dsmexec` qui se contente de les diffuser mais ne les utilise pas.** Ce fonctionnement induit une indépendance fonctionnelle entre les applications utilisant la bibliothèque `libdsm` et le programme lanceur `dsmexec`. Par exemple, ces données pourraient être modifiées dans une version ultérieure de la bibliothèque `libdsm` sans que le programme lanceur n'ait besoin d'être révisé.

3 — Les échanges ne sont pas systématiques La section 4.3.1 précise que le programme `dsmexec` doit être capable de lancer n'importe quel type de programme. Ainsi, il est possible de déployer des processus qui n'auront pas besoin de mettre en place ces interconnexions. Dans un tel cas, il n'est pas utile de procéder à l'échange des informations nécessaires. Autrement dit, **l'échange des informations d'interconnexion n'aura lieu que si l'application déployée utilise la bibliothèque `libdsm`.** Cela signifie aussi que la socket ouverte par `dsmexec` pour effectuer les échanges n'est pas utile dans le cas où les programmes n'utilisent pas la `libdsm`.

4 — Représentation des donnnées dans `dsmexec` Afin de respecter le principe d'indépendance énoncé ci-dessus, les informations d'interconnexion sont caractérisées par les deux champs suivants dans la structure `struct remote_proc` (cf code fourni) :

```
struct remote_proc {

  /* ... */

  void   *connect_info;      /* données -> pointeur générique  */
                             /*            car nature inconnue */
  size_t  connect_info_size; /* taille (en octet) des données  */

  /* ... */

}
```

5 — Protocole d'échange des donnnées Le principe d'indépendance repose également sur la mise en place d'un **protocole d'échange de ces données d'interconnexion**. Ce protocole d'échange suit les étapes suivantes du côté du programme `dsmexec` (ces étapes 1 à 4 sont à effectuer pour chaque processus distant) :

Étape 1 : récupération du numéro du processus distant (numéro codé avec un entier de type `int`);

Étape 2 : récupération de la taille des informations de connexion (et stockage dans le champ `connect_info_size`). Cette information est de type `size_t` (cf structure ci-dessus);

Étape 3 : allocation de la mémoire nécessaire au stockage du champ `connect_info` (de la structure `struct remote_proc`) dont la taille est maintenant connue (précision : la taille de cette structure de données est identique pour tous les processus distants).

Étape 4 : récupération des informations proprement dites, avec la taille adéquate reçue à l'étape 2.

Une fois que toutes ces informations de connexion sont disponibles du côté du `dsmexec` (ie il a effectué n fois l'étape 4), ce dernier envoie l'ensemble des données à tous les processus applicatifs. Pour la diffusion de ces données, le processus `dsmexec` se contentera de les envoyer directement, c'est-à-dire qu'il enverra n structures à chaque processus (donc n fois au total).

6 — Un programme de test Afin de tester si votre implémentation du protocole d'échange est correcte, vous allez utiliser le programme de test appelé `data_exchange`, mis à disposition dans le répertoire `binaries`. Ce programme implémente ce protocole, mais uniquement la partie des processus applicatifs.

Attention, ce programme utilise les quatre variables d'environnement suivantes (cf section 4.4) :

1. Le nombre de processus : `DSMEXEC_RANKNUM`
2. Le numéro du processus distant : `DSMEXEC_RANKID`
3. Le nom de la machine d'exécution de dsmexec : `DSMEXEC_HOSTNAME`
4. Le numéro de port de la socket d'échange de dsmexec : `DSMEXEC_PORTNUM`

Vous remarquerez que les deux dernières variables d'environnement contiennent les informations nécessaires aux processus distants pour se connecter au processus `dsmexec`. Quant aux deux premières, elles vont servir à initialiser des variables de l'interface de la bibliothèque `libdsm` (cf Section 5.1.2).

Dans le cas de 4 processus distants, tous lancés sur la même machine appelée Palamède, l'affichage de ce programme est par exemple :

```
[Proc 1 : Palamede : stdout]: Info from dsmexec : I'm process 1 / 4 processes total
[Proc 1 : Palamede : stdout]: Struct size is 136
[Proc 1 : Palamede : stdout]: [1] got connection info from dsmexec
[Proc 1 : Palamede : stdout]: [1] Process 0 is on machine: Palamede (port #: 36693)
[Proc 1 : Palamede : stdout]: [1] Process 1 is on machine: Palamede (port #: 55565)
[Proc 1 : Palamede : stdout]: [1] Process 2 is on machine: Palamede (port #: 60773)
[Proc 1 : Palamede : stdout]: [1] Process 3 is on machine: Palamede (port #: 35757)
[Proc 0 : Palamede : stdout]: Info from dsmexec : I'm process 0 / 4 processes total
[Proc 0 : Palamede : stdout]: Struct size is 136
[Proc 0 : Palamede : stdout]: [0] got connection info from dsmexec
[Proc 0 : Palamede : stdout]: [0] Process 0 is on machine: Palamede (port #: 36693)
[Proc 0 : Palamede : stdout]: [0] Process 1 is on machine: Palamede (port #: 55565)
[Proc 0 : Palamede : stdout]: [0] Process 2 is on machine: Palamede (port #: 60773)
[Proc 0 : Palamede : stdout]: [0] Process 3 is on machine: Palamede (port #: 35757)
[Proc 2 : Palamede : stdout]: Info from dsmexec : I'm process 2 / 4 processes total
[Proc 2 : Palamede : stdout]: Struct size is 136
[Proc 2 : Palamede : stdout]: [2] got connection info from dsmexec
[Proc 2 : Palamede : stdout]: [2] Process 0 is on machine: Palamede (port #: 36693)
[Proc 2 : Palamede : stdout]: [2] Process 1 is on machine: Palamede (port #: 55565)
[Proc 2 : Palamede : stdout]: [2] Process 2 is on machine: Palamede (port #: 60773)
[Proc 2 : Palamede : stdout]: [2] Process 3 is on machine: Palamede (port #: 35757)
[Proc 3 : Palamede : stdout]: Info from dsmexec : I'm process 3 / 4 processes total
[Proc 3 : Palamede : stdout]: Struct size is 136
[Proc 3 : Palamede : stdout]: [3] got connection info from dsmexec
[Proc 3 : Palamede : stdout]: [3] Process 0 is on machine: Palamede (port #: 36693)
[Proc 3 : Palamede : stdout]: [3] Process 1 is on machine: Palamede (port #: 55565)
[Proc 3 : Palamede : stdout]: [3] Process 2 is on machine: Palamede (port #: 60773)
[Proc 3 : Palamede : stdout]: [3] Process 3 is on machine: Palamede (port #: 35757)
```

Veuillez noter que dans votre cas, la taille de la structure pourra être différente, tout comme les numéros de port ainsi que le formatage et l'ordre d'affichage.

4.3.4 Schéma de fonctionnement du lanceur

Les liens entre les différentes entités (processus locaux, distants, etc.) ainsi que les différents rôles du programme `dsmexec` sont résumés sur la figure 1, qui montre un exemple avec le déploiement de trois processus distants sur les machines `toto`, `tata` et `titi`.

Rappel : une fois le déploiement effectué, le processus `dsmexec` attend que des données (des *caractères*) arrivent sur les tubes dédiés aux flux sur `stdout` et `stderr` et les affiche au fur et à mesure de leur disponibilité, avec un formatage correspondant à celui de l'exemple ci-dessus.

4.4 Hypothèses concernant les processus déployés avec `dsmexec`

Les processus déployés avec le lanceur `dsmexec` suivent les principes suivants :

— contexte **statique** : le nombre de processus déployés ne change pas en cours d'exécution. Plus précisément, on ne rajoute pas de nouveau processus et aucun processus ne quitte l'application en cours

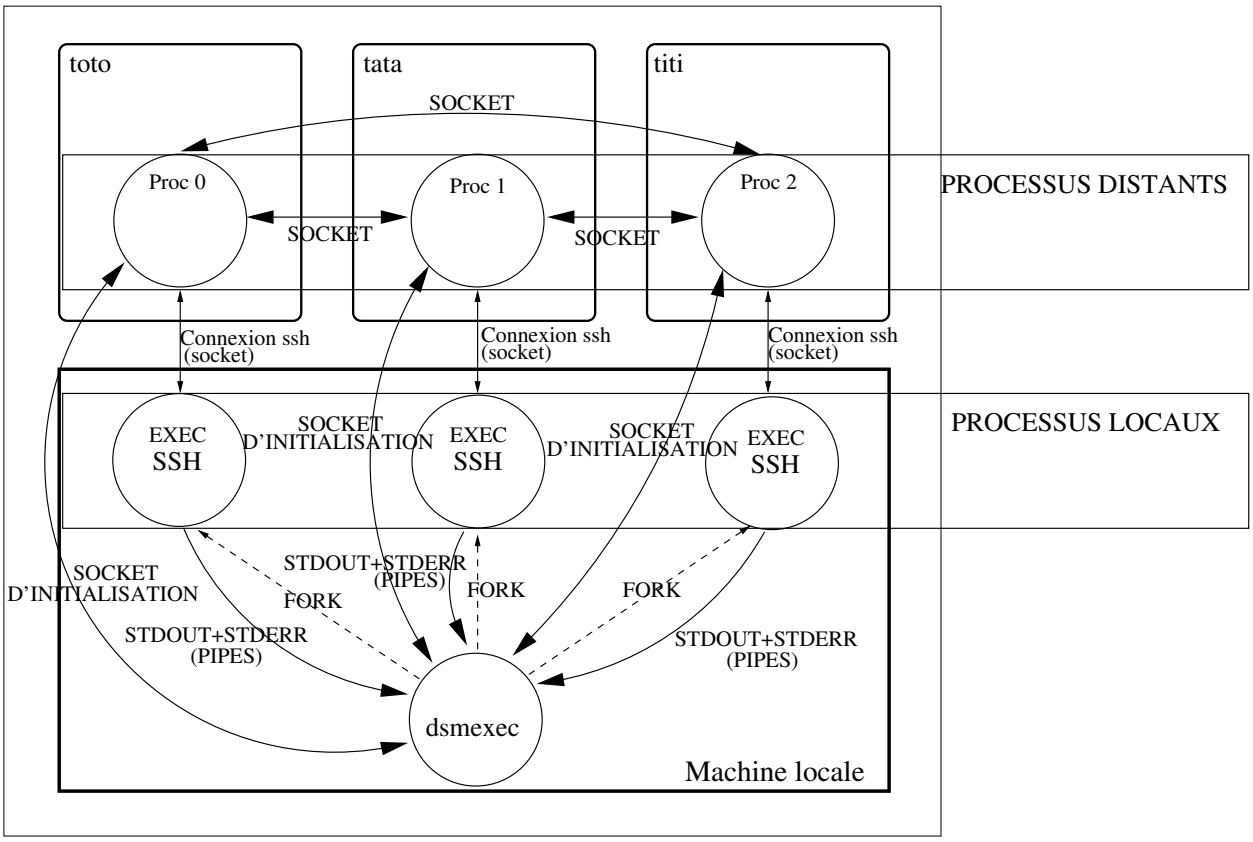


FIGURE 1 – Interactions entre processus distants, processus locaux et `dsmexec`

de route ;

— les processus distants sont identifiés par un numéro unique qui n'a de sens qu'au sein d'une application donnée et n'a aucun rapport avec le pid du processus ;

— tout processus distant peut connaître son numéro grâce à la **variable d'environnement** `DSMEXEC_RANKID` ;

— tout processus distant peut connaître le nombre total de processus déployés grâce à la **variable d'environnement** `DSMEXEC_RANKNUM` ;

— les numéros des processus distants sont contigus et compris entre 0 et `DSMEXEC_RANKNUM`−1 ;

— tous les processus ont accès à un système de fichier commun et partagé. La question de la disponibilité sur les différentes machines du programme exécutable à lancer ne se posera donc pas.

4.5 Organisation du matériel de départ

Le matériel fourni pour cette première phase est localisé dans le répertoire `Phase1` dans vos dépôts Git respectifs. Ce répertoire contient les éléments suivants :

```
- binaries
  |____ data_exchange ------------> | programme de test pour vérifier que
  |                                 | le protocole d'échange des infos de
  |                                 | connexion est correctement implémenté
  |                                 | côté dsmexec (copié dans bin à
  |                                 | l'installation)
  |____ libdsmexec_correction.a -> | bibliothèque contenant des versions
                                    | corrigées de certaines fonctions
                                    | (copiée dans libs à l'installation)

- dsmexec.c ----------------------> | squelette du programme dsmexec (main)
                                    | décomposé en étapes principales

- examples -----------------------> | programmes d'exemple utilisables
  |____ basic.c                    | (vous rajouterez vos propres
  |____ calcul.c                   | programmes d'exemple dans ce répertoire)
  |____ hello.c
  |____ Makefile
  |____ trick_hello.c

- include ------------------------> | fichiers d'en-tête avec les définitions
  |____ common_services.h          | de constantes, de structures de données et
  |____ correction.h               | de prototypes de fonctions

- libs --------------------------->| bibliothèques utilisées pour la construction
  |____ libdsmexec_correction.a    | de l'exécutable dsmexec
  |____ libdsmexec_services.a

- machine_file ------------------> | fichier de machines d'exemple

- machines_list.txt -------------> | liste de noms de machines (par salle)

- Makefile ----------------------> | Makefile principal pour compiler la Phase 1
                                    | utilisant tous les Makefiles des répertoires
- services
  |____ childproc_management.c --> | gestion des processus-enfants
  |____ cleanup_management.c ----> | libération des ressources globales
  |____ file_management.c -------> | lecture du fichier de machines
  |____ io_management.c ---------> | gestion des tubes et des entrées/sorties
  |____ Makefile
  |____ network_management.c ----> | fonctions reseau
  |____ spawn_management.c ------> | création des processus-enfants + gestion de ssh
```

4.5.1 Organisation du fichier `dsmexec.c`

Le fichier `dsmexec.c` contient le squelette du lanceur et est décomposé en cinq étapes principales (plus ou moins) successives. Ces étapes sont :

Étape 1 : lecture du fichier de machines et récupération des informations importantes : nombre total de processus à déployer et noms des machines distantes. (cf exemple 4.1)

Étape 2 : création de la socket d'écoute pour les échanges de données avec les processus distants. Cette étape est optionnelle en fonction du type du programme déployé.

Étape 3 : création des processus-enfants locaux et des processus distants avec ssh; création des tubes de communications avec les processus-enfants locaux et redirections des flux `stdout` et `stderr` des processus-enfants. Le processus `dsmexec` — quant à lui — ne redirige rien du tout. Cette étape inclut également la gestion de la terminaison des processus-enfants (fonction `childprocs_mgmt_setup`). Cependant, cette fonctionnalité est dans une fonction séparée car elle peut-être appelée à différents endroits en fonction de la solution de gestion choisie.

Étape 4 : gestion des entrées-sorties avec les tubes; récupération de l'affichage des processus-distants et affichage centralisé formaté.

Étape 5 : terminaison du programme et libération des ressources utilisées

4.5.2 Fonctions disponibles dans la bibliothèque `libdsmexec_correction.a`

Pour certaines de ces étapes principales, une fonction implémentant le service est d'ores-et-déjà fournie :

Étape 1 : fonction `file_management_correction`
Étape 2 : fonction `socket_creation_correction`
Étape 3 : fonction `spawn_local_procs_correction`
Étape 4 : fonction `wait_for_all_io_correction`

Ces fonctions vous permettent d'implémenter n'importe quelle étape sans dépendre de la réalisation d'une étape précédente. Ainsi, il sera possible d'effectuer le travail dans l'ordre que vous souhaitez.

En fonction de votre implémentation de la gestion des processus-enfants zombies, il est possible que la fonction `wait_for_all_io_correction` bloque et que l'exécution d'un programme ne se termine pas correctement.

Tous les prototypes de ces fonctions se trouvent dans le fichier `include/correction.h`. Merci de vous y référer pour plus de précisions et comprendre leur fonctionnement.

4.5.3 Compilation des sources

Pour compiler les sources, vous **devez** utiliser le fichier `Makefile` fourni. Cette compilation s'effectue en tapant la commande `make install` qui :

1. crée un répertoire supplémentaire appelé `bin`, contenant tous les programmes exécutables : le lanceur `dsmexec` ainsi que tous les programmes d'exemple présents dans le répertoire `examples`. Ce répertoire `bin` est recréé chaque fois qu'il est effacé (par exemple avec la commande `make distclean`).
2. crée un répertoire supplémentaire appelé `libs` qui contient les bibliothèques nécessaires pour créer l'exécutable `dsmexec`. La bibliothèque `libdsmexec_correction.a` est notamment copiée depuis le répertoire `binaries` dans ce répertoire `libs`.
3. copie le programme `data_exchange` depuis le répertoire `binaries` vers le répertoire `bin`

Notez que le répertoire `binaries` **n'est pas modifié** quand vous exécutez la commande `make clean` ou `make distclean`.

4.6 Éléments modifiables et non-modifiables

1. Afin de garantir que votre programme `dsmexec` soit compatible avec les fonctions corrigées fournies dans la bibliothèque `libdsmexec_correction`, certains éléments ne sont pas modifiables *a priori* (cf les commentaires dans le fichier `include/common_services.h` :
   — la constante `MAX_STR` n'est pas modifiable ;
   — le début de la structure `struct remote_proc` n'est pas modifiable.
   ⇒ Si vous faites le choix de ne **pas** utiliser les fonctions corrigées alors les deux éléments listés ci-dessus deviennent modifiables.
2. l'**organisation et la structure du code-source ne sont pas modifiables** : il ne faut pas changer les noms des fichiers et répertoires fournis, ni les déplacer.
3. Les fichiers `Makefile`, et en particulier les **flags de compilation ne sont pas modifiables** dans le but de rendre cette compilation plus permissive.
4. Le nom de la variable d'environnement `DSM_BIN` est **imposé** (cf section 7.3).

Dans le doute, sollicitez vos encadrants pour déterminer si ce que vous souhaitez modifier l'est effectivement ou non. . .

5 Phase 2 : implémentation de la bibliothèque `libdsm`

Une fois votre lanceur `dsmexec` opérationnel, vous allez mettre en place la bibliothèque `libdsm` qui va permettre à vos programmes d'utiliser le mécanisme décrit en section 3.

Organisation de la mémoire partagée

Les propriétés des pages de la mémoire partagée sont les suivantes :

- le nombre de pages est limité à `PAGE_NUMBER` qui est une constante *modifiable* ;
- chaque page mémoire est identifiée par un numéro. Cette numérotation est linéaire et contigüe avec des valeurs comprises entre 0 et `PAGE_NUMBER`−1) ;
- la taille d'une page est de `PAGE_SIZE` octets, qui est une constante *non modifiable* ;
- la plage des adresses considérée sera comprise entre `BASE_ADDR` et `TOP_ADDR`. **Cette plage d'adresses est identique pour tous les processus de l'application et n'est pas modifiable** ;
- tous les processus sont connectés les uns avec les autres via des sockets TCP ;
- chaque processus dispose d'une table stockant les informations sur les pages mémoire (par exemple le numéro du propriétaire de la page) ;
- l'allocation *initiale* des pages est faite cycliquement (cf figure 2 ci-dessous).

La figure 2 montre un exemple d'allocation de départ pour 6 pages réparties entre 3 processus. Le processus 0 possède les pages 0 et 3, le processus 1 possède les pages 1 et 4 et le processus 2 possède les pages 2 et 5. Cependant, chaque processus peut accéder à l'ensemble des pages : par exemple, si le processus 0 accède à la page 0 ou 3, les accès se font normalement. Si le processus 0 accède à une autre page, cela provoquera un `SEGFAULT` et le processus enverra une requête au processus 1 (resp. 2) pour récupérer la page 1 ou 4 (resp. 2 ou 5).

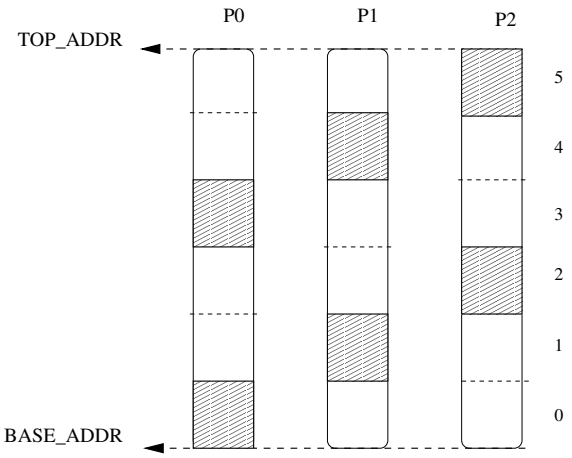


FIGURE 2 – Répartition *initiale* des pages mémoire

5.1 Interface de la bibliothèque

L'interface de la bibliothèque `libdsm` est réduite et comprend deux fonctions et deux variables.

5.1.1 Fonctions

Les deux fonctions à implémenter sont les suivantes :

1. `char *dsm_init(void);`
   La fonction `dsm_init` renvoie un pointeur : c'est le pointeur marquant le début de la zone de mémoire distribuée. **Par ailleurs, la fonction `dsm_init` initialise à `0` l'ensemble des pages mémoire allouées.**
2. `void dsm_finalize(void);`
   La fonction `dsm_finalize` permet de libérer les ressources éventuellement allouées par la bibliothèque au moment de l'initialisation.

5.1.2 Variables

Également, cette interface repose sur deux variables **globales** utilisables dans les applications :

1. `dsm_node_id` : qui représente le **numéro** d'un processus dans l'application distribuée ;
2. `dsm_node_num` : qui représente le **nombre** de processus de l'application distribuée.

5.2 Travail à effectuer

La suite du travail va consister à :

— mettre en place le bon traitant de signal pour rattraper les erreurs de segmentation (conseil : lisez bien le manuel pour la fonction `sigaction`)

— mettre en place les envois/réceptions de requêtes. En ce qui concerne les requêtes, nous pouvons en identifier au moins quatre types :

1. les requêtes pour demander une page manquante ;
2. les requêtes pour envoyer une page à un processus demandeur ;
3. les requêtes pour indiquer qu'un processus est rentré en phase de terminaison du programme (i.e il est dans la fonction `dsm_finalize`) ;
4. les requêtes de mise à jour des informations de la table des pages.

— garantir que les informations concernant les pages (état, propriétaire, etc.) sont bien bien cohérentes d'un processus à l'autre.

5.3 Organisation du matériel de départ

Dans cette phase également, une ébauche de code est fournie pour commencer à travailler. **Lisez attentivement les commentaires présents dans le fichier `dsm.c`, en particulier ceux dans la fonction `dsm_init`.**

Le matériel fourni pour cette seconde phase est localisé dans le répertoire `Phase2` dans vos dépôts Git respectifs. Ce répertoire contient les éléments suivants :

```
- dsm.c ---------------------> | squelette des  fonctions de la libdsm

- dsm.h ---------------------> | interface de la libdsm (non-modifiable)

- examples ------------------> | programmes d'exemple utilisables
  |____ double_touch.c         | (vous rajouterez vos propres
  |____ all_touch.c            | programmes d'exemple dans ce répertoire)

- include -------------------> | fichiers d'en-tête internes pour l'implémentation
  |____ dsm_impl.h             | de la libdsm. Les programmes utilisant la libdsm
                               | n'ont pas accès aux définitions dans ce fichier

- Makefile ------------------> | Makefile pour compiler la bibliothèque et les exemples

- services
  |____ network.c ------------> | fonctions réseau utilisées en interne dans la libdsm
  |____ page_management.c ---> | fonctions de gestion des pages-mémoire
```

Le matériel fourni est composé des fichiers suivants :

— un fichier `dsm.h`, qui contient les prototypes des fonctions de la `libdsm` utilisées par les programmes d'exemple. C'est dans ce fichier que se trouve l'*interface* de la bibliothèque de DSM.

— un fichier `dsm.c` qui contient l'implémentation des fonctions de DSM qui peuvent être utilisées par les programmes d'exemple. Au besoin, vous pourrez y déclarer des fonctions non directement utilisables par les programmes d'exemple (i.e des fonctions `static`). Vous allez donc travailler essentiellement sur ce fichier `dsm.c` pour compléter cette phase du projet.

— Un `Makefile` permettant de compiler (commande : `make` tout court) à la fois la bibliothèque `libdsm` ainsi que les programmes d'exemple dont les sources sont localisées dans le répertoire `examples`. **Attention : les exécutables seront déplacés dans le répertoire `$DSM_BIN`, c'est-à-dire dans le répertoire `bin` du code fourni lors de la Phase 1.**

Par ailleurs, **une version fonctionnelle de dsmexec vous est fournie** : l'exécutable se trouve dans le répertoire `Phase1/binaries` de votre dépôt Git. Cette version possède quelques options qui pourront éventuellement vous être utiles pour cette seconde phase (tapez la commande `dsmexec -h` pour obtenir la liste des options actuellement supportées).

5.4 Protocole d'échange des données de connexion

La section 4.3.3 décrit le protocole d'échange des données de connexion du côté du processus `dsmexec`. Du côté des processus applicatifs de la dsm, l'échange des données de connexion suit les étapes suivantes :

Étape 1 : envoi de son numéro de processus distant (numéro codé avec un entier de type `int`). Cette étape correspond à l'étape 1 du protocole décrit en 4.3.3.

Étape 2 : envoi de la taille de la structure de données de connexion. Cette taille est de type `size_t`. Cette étape correspond à l'étape 2 du protocole décrit en 4.3.3.

Étape 3 : envoi de la structure proprement dite (de type `proc_conn_info_t`) contenant les informations de connexion du processus. Cette étape correspond à l'étape 4 du protocole décrit en 4.3.3.

Étape 4 : récupération des informations de connexion proprement dites : chaque processus récupère `dsm_node_num` structures de type `proc_conn_info_t`. Cette récupération est plus simple dans le sens `dsmexec` → processus distants que dans l'autre sens car ni le numéro du processus émettant ces données, ni leur taille ne sont nécessaires car :
- le processus `dsmexec` ne possède pas de numéro (il ne fait pas partie de l'application à déployer)
- les données d'interconnexion ne sont **pas** opaques pour les processus distants (ils connaissent la définition de la structure de données utilisée).

5.5 Exemple : le programme `double_touch`

Examinez le programme suivant (dont on suppose qu'il s'exécute sur des machines différentes) :

```
#include "dsm.h"

int main(int argc, char **argv)
{
  char *pointer = dsm_init();
  char *current = pointer;
  int   value;

  fprintf(stdout,"[Proc %i] base address for shared mapping is: %p\n",
                 dsm_node_id, pointer);

  if(0 == dsm_node_id){
    *((int *)current) += 4;
    value = *((int *)current);
    printf("[Proc %i] integer value: %i\n", dsm_node_id, value);
  } else if(1 == dsm_node_id){
    *((int *)current) += 8;
    value = *((int *)current);
    printf("[Proc %i] integer value: %i\n", dsm_node_id, value);
  } else {
    printf("[Proc %i] I'm not part of this \n", dsm_node_id);
  }

  fprintf(stdout,"[Proc %i] Finalizing ...", dsm_node_id);
  dsm_finalize();
  fprintf(stdout,"done\n");

  return 1;
}
```

Le fonctionnement de ce programme est le suivant : au départ, toute la mémoire est initialisée à 0, donc le premier entier située dans la première page vaut 0. Ensuite le processus 0 rajoute 4 et le processus 1 rajoute 8. Chaque processus affiche la valeur de cet entier. Selon l'ordre d'exécution du programme, on aura l'affichage 4 (processus 0) et 12 (processus 1) ou bien 12 (processus 0) et 8 (processus 1). Il est évident que ce programme pose problème puisque l'exécution peut mener à des résultats différents selon les processus. Ce qui importe dans cet exemple, c'est que la variable contienne la valeur 12 au final. **Nous ne vous demandons pas de gérer ce problème dans le cadre de ce projet.**

6 Planning indicatif

6.1 Division en tâches et étapes

Phase 1

Étape 0 : Configuration des dépôts Git, de ssh et définition des variables d'environnement
Étape 1 : Gestion du fichier de machines

1. lecture pour déterminer le nombre de processus à lancer
2. relecture pour récupérer les noms d'hôtes

⇒ Cette étape fait l'objet d'un exercice de Prog. Sys

Étape 2 : Création de la socket maître/serveur pour les échanges d'information
⇒ Cette étape a été traitée (au moins partiellement) en cours de programmation réseau
Étape 3 : Lancement des processus locaux et création des processus distants

1. fork des processus locaux + gestion des zombies
2. création des tubes + redirection de STDOUT + STDERR
3. création de la ligne de commande ssh (en particulier les variables d'environnement)
4. protocole d'échange de données avec `dsm_init`

Étape 4 : Boucle de récupération des outputs sur les tubes STDOUT + STDERR

1. récupération des données
2. formatage des données avec un formatage adéquat

Étape 5 : Récupération des ressources + cleanup

Phase 2

Étape 1 : Initialisation du système : interconnexion des processus distants
Étape 2 : Traitant du signal `SIGSEGV`
Étape 3 : Thread de communication et gestion des requêtes
Étape 4 : Finalisation du programme (protocole et synchronisation avec le thread démon)

6.2 Nombre de séances

Rappel : 11 séances de 1h20 sont consacrées pour la Phase 1 et 9 séances pour la Phase 2.

Fin de Phase 1 : Lundi 23 Novembre 2026

Fin de Phase 2 : Lundi 14 Décembre 2026

Phase	Étape	Nombre de séances (1h20)
1	0	1
1	1 + 5	1 ou 2
1	2	1 ou 2
1	3	4 ou 5
1	4	2 ou 3
2	1	2
2	2	1
2	3	4 ou 5
2	4	1 ou 2

7 Travail de la première séance

Pour travailler dans des conditions correctes, il est indispensable que les trois tâches suivantes soient accomplies lors de la toute première séance de projet.

7.1 Tâche 1 : configuration des dépôts Git

La plateforme Thor sera utilisée pendant toute la durée du projet. Un dépôt Git y est créé pour chacun(e). Une documentation sur Git/Thor est disponible à :

`https://thor.enseirb-matmeca.fr/ruby/docs/repository/git`.

Il est de votre ressort de configurer correctement vos dépôts pour que les opérations de type «push» soient acceptées. Deux options de configurations s'offrent à vous :

1. Configuration globale : si vous optez pour cette option, il faut éditer le fichier `$HOME/.gitconfig`
2. Configuration locale : si vous optez pour cette option, il faut éditer le fichier `.git/config`, sachant que le répertoire `.git` se trouve **à la racine du dépôt GIT particulier que vous voulez configurer.**

Dans tous les cas, il faut rajouter votre nom et une adresse email **valide**, c'est-à-dire avec un nom de domaine `enseirb-matmeca.fr` ou `bordeaux-inp.fr` de la façon suivante :

```
[user]
   name = Xxxx Yyyy
   email = xxxx.yyyy@bordeaux-inp.fr
```

7.2 Tâche 2 : configuration de `ssh`

Pour mener à bien ce projet, il vous sera **indispensable** d'être en capacité de vous connecter d'**une machine de l'école à une autre machine de l'école**, avec la commande `ssh` mais sans que la commande `ssh` n'affiche quoi que ce soit. Deux situations sont possibles :

1. Vous possédez déjà une clef SSH pour vous connecter d'une machine de l'école à une autre. Dans ce cas, vous avez juste besoin de configurer ssh pour répondre aux besoins du projet (cf ci-dessous). Également, vous pouvez générer une nouvelle clef si vous le désirez. Il sera possible par la suite de choisir la clef `ssh` à utiliser via l'option `-i` de la commande `ssh`. Si votre clef ssh ne possède pas de passphrase, vous **devez** en rajouter une avec la commande `ssh-keygen -p`.
2. Si vous ne possédez pas une telle clef SSH, alors il vous faudra en générer une. Des informations sont disponibles à cet URL `https://thor.enseirb-matmeca.fr/ruby/docs/teaching/authentication`

Une fois laclef créée, il ne faut pas oublier de rajouter la partie **publique** de cette clef dans la liste des clefs autorisées. Par exemple, si je crée un couple de clefs `ma_jolie_clef` (partie privée) et `ma_jolie_clef.pub` (partie publique), cet ajout sera fait ainsi :

```
cat ma_jolie_clef.pub >> $HOME/.ssh/authorized_keys
```

Dans tous les cas, il vous faut aussi configurer `ssh` afin que les connexions se fassent de façon silencieuse (i.e sans que la commande `ssh` n'affiche quoi que ce soit au moment d'une demande de connexion). Pour se faire, les éléments ci-dessous sont à rajouter dans votre fichier `$HOME/.ssh/config` sur présent sur votre compte ENSEIRB-MMK.

```
# Permet d'avoir les FQDN des machines
CanonicalDomains pedago.ipb.fr
CanonicalizeHostname yes
CanonicalizeFallbackLocal yes

AddKeysToAgent yes

Host *.pedago.ipb.fr
ForwardAgent yes
StrictHostKeyChecking no #ou accept-new
ForwardX11 yes
```

Nous ne vous demandons pas de créer des clefs SSH entre votre machine personnelle et les machines de l'école.

7.3 Tâche 3 : définition des variables d'environnement pour le déploiement d'applications

Vous devez maintenant définir une variable d'environnement appelée **`DSM_BIN` dont le nom est imposé**. Cette variable contient le nom du chemin vers un répertoire appelé `bin` dans les sources de votre projet où seront copiés tous les exécutables et programmes de test compilés (aussi bien pour la phase 1 que pour la phase 2). Cette définition va vous permettre d'éviter d'avoir à donner les noms absolus des programmes d'exemple et de la commande `dsmexec`. Également les commandes exécutées par `ssh` auront pour répertoire de travail votre `$HOME` : la commande `ssh` ne pourra pas trouver le programme à déployer en cas d'utilisation de noms relatifs (plus courts). La définition et l'utilisation de `DSM_BIN` permet de régler cet autre problème.

Afin que tout fonctionne correctement (ie afin que le système trouve tout seul le bon chemin vers les exécutables du projet), le chemin désigné par `DSM_BIN` **doit également être copié** dans la variable d'environnement `PATH`.

Pour ce faire tout ceci, vous devez éditer votre fichier `$HOME/.bashrc` et y placer les définitions suivantes :

```
export DSM_BIN=/mon/chemin/vers/mon/projet/bin
export PATH=$DSM_BIN:$PATH
```

N'oubliez pas de sourcer votre fichier `$HOME/.bashrc` après l'avoir édité !

Vous pouvez tester ensuite si la manœuvre a fonctionné en tapant les commandes :

```
echo $DSM_BIN
```

ou

```
which dsmexec  # si vous avez déjà compilé dsmexec
```

afin de vérifier que votre définition des chemins est correcte.

— ❄ —